In [4]:
```
import pandas as pd
import numpy as np

# Load Excel file
path = "Ilmhona_Funder_Outreach_Tracker.xlsx"

tracker = pd.read_excel(
    path,
    sheet_name="Funder Outreach Tracker",
    skiprows=2
)


tracker.columns = [
    "funder",
    "activity",
    "date_range",
    "lead_contact",
    "engagement_type",
    "outcome_status",
    "reengage",
    "suggested_next_step"
]

tracker = tracker.dropna(subset=["funder"]).copy()

# Combine text fields
tracker["text"] = (
    tracker["funder"].fillna("") + " " +
    tracker["activity"].fillna("") + " " +
    tracker["engagement_type"].fillna("") + " " +
    tracker["outcome_status"].fillna("") + " " +
    tracker["suggested_next_step"].fillna("")
).str.lower()

# Basic flags from past data
tracker["past_funder"] = tracker["text"].str.contains("funded|signed agreeme
tracker["warm_contact"] = tracker["text"].str.contains("warm|re-engage|follo
tracker["proposal_submitted"] = tracker["text"].str.contains("proposal|conce
tracker["stalled"] = tracker["text"].str.contains("stalled|no decision|under
tracker["implementation_partner"] = tracker["text"].str.contains("implementi

# Theme flags
tracker["digital_skills"] = tracker["text"].str.contains("digital|skills|ict
tracker["youth_women"] = tracker["text"].str.contains("youth|women|girls|stu
tracker["workforce"] = tracker["text"].str.contains("employment|workforce|vo
tracker["regional_expansion"] = tracker["text"].str.contains("bokhtar|istara

# Convert existing HIGH/MED/LOW into base score
def reengage_score(value):
    value = str(value).lower()
    if "high" in value:
        return 10
    elif "medium" in value:
```

```
        return 6
    elif "low" in value:
        return 2
    else:
        return 0

tracker["base_reengage_score"] = tracker["reengage"].apply(reengage_score)

# Ilmhona funder-fit score
def score_funder(row):
    score = row["base_reengage_score"]

    if row["past_funder"]:
        score += 5
    if row["warm_contact"]:
        score += 4
    if row["proposal_submitted"]:
        score += 3
    if row["implementation_partner"]:
        score += 3
    if row["digital_skills"]:
        score += 3
    if row["youth_women"]:
        score += 3
    if row["workforce"]:
        score += 3
    if row["regional_expansion"]:
        score += 2
    if row["stalled"]:
        score += 1  # not bad; means possible revival

    return score

tracker["ilmhona_fit_score"] = tracker.apply(score_funder, axis=1)

# Recommendation labels
def recommendation(score):
    if score >= 25:
        return "Very strong fit — prioritize soon"
    elif score >= 18:
        return "Strong fit — worth follow-up"
    elif score >= 12:
        return "Moderate fit — monitor or research more"
    else:
        return "Lower priority"

tracker["recommendation"] = tracker["ilmhona_fit_score"].apply(recommendatic

#results
output = tracker[[
    "funder",
    "activity",
    "outcome_status",
    "reengage",
    "suggested_next_step",
    "ilmhona_fit_score",
```

```
    "recommendation"
]].sort_values("ilmhona_fit_score", ascending=False)

print(output.to_string(index=False))

# Save result
output.to_csv("ilmhona_funder_fit_scores.csv", index=False)
tracker.to_csv("ilmhona_funder_scoring_full_data.csv", index=False)
```

| | | |

```
17 Moderate fit — monitor or research more
                                                                IOM Tajikistan
Concept note (template received Mar 2025); Bokhtar courses — Digital Literac
y, Digital Freelance, etc.                          ENGAGED — IOM template rec
eived but full concept not yet submitted in inbox YES — MEDIUM  Submit IOM c
oncept note using their March 2025 template; introduce yourself to Anne's su
ccessor.                 14 Moderate fit — monitor or research more
                     USAID Tajikistan (Coalition of Employers visit)
USAID donor visit + employer coalition introduction
WARM CONTACT — no formal application on file    YES — LOW        Identify cu
rrent USAID/CA contact (post-2025 reorg) and pitch a workforce-development L
OI.                 14 Moderate fit — monitor or research more
```

In [11]:
```python
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

historical_path = "historical_funders_features.csv"
new_funders_path = "ilmhona_potential_funders_scoring_input.xlsx"

historical = pd.read_csv(historical_path)
new_funders = pd.read_csv(new_funders_path)

# Clean column names
historical.columns = historical.columns.str.strip()
new_funders.columns = new_funders.columns.str.strip()

# Creating reengage_high if it doesn't already exist
if "reengage_high" not in historical.columns:
    if "reengage" in historical.columns:
        historical["reengage_high"] = (
            historical["reengage"]
            .astype(str)
            .str.lower()
            .str.contains("high")
            .astype(int)
        )
    else:
        historical["reengage_high"] = 0

features = [
    "digital_skills",
    "youth",
    "women",
    "workforce",
    "entrepreneurship",
    "tajikistan",
    "central_asia",
    "government",
    "foundation",
    "un_agency",
    "development_agency"
]

# Add any missing feature columns as 0
for col in features + ["funded_before", "reengage_high"]:
```

```
    if col not in historical.columns:
        historical[col] = 0
    if col in features and col not in new_funders.columns:
        new_funders[col] = 0

strong_historical = historical[
    (historical["funded_before"] == 1) |
    (historical["reengage_high"] == 1)
].copy()

# Backup in case no rows qualify
if strong_historical.empty:
    strong_historical = historical.copy()

historical_matrix = strong_historical[features].fillna(0)
new_matrix = new_funders[features].fillna(0)

similarity_matrix = cosine_similarity(new_matrix, historical_matrix)

new_funders["strategic_fit_score"] = similarity_matrix.mean(axis=1)

closest_match_index = similarity_matrix.argmax(axis=1)

new_funders["closest_historical_match"] = [
    strong_historical.iloc[i]["funder"]
    for i in closest_match_index
]

new_funders["closest_match_score"] = similarity_matrix.max(axis=1)

def recommendation(score):
    if score >= 0.75:
        return "Very strong fit — prioritize soon"
    elif score >= 0.60:
        return "Strong fit — worth follow-up"
    elif score >= 0.45:
        return "Moderate fit — research more"
    else:
        return "Lower fit"

new_funders["recommendation"] = new_funders["strategic_fit_score"].apply(rec

def create_rationale(row):
    reasons = []

    if row["digital_skills"] == 1:
        reasons.append("digital skills")
    if row["youth"] == 1:
        reasons.append("youth")
    if row["women"] == 1:
        reasons.append("women/gender inclusion")
    if row["workforce"] == 1:
        reasons.append("workforce development")
    if row["entrepreneurship"] == 1:
        reasons.append("entrepreneurship")
    if row["tajikistan"] == 1:
```

```
        reasons.append("Tajikistan focus")
    elif row["central_asia"] == 1:
        reasons.append("Central Asia focus")

    if reasons:
        return "Aligned on: " + ", ".join(reasons)
    return "Limited direct alignment based on current features"

new_funders["rationale"] = new_funders.apply(create_rationale, axis=1)

recommendations = new_funders.sort_values("strategic_fit_score", ascending=F

output = recommendations[[
    "funder",
    "strategic_fit_score",
    "recommendation",
    "closest_historical_match",
    "closest_match_score",
    "rationale"
]]

output.to_csv("new_funder_recommendations.csv", index=False)
recommendations.to_csv("new_funder_scoring_full.csv", index=False)

print(output.to_string(index=False))
```

```
---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
Cell In[11], line 9
      6 new_funders_path = "ilmhona_potential_funders_scoring_input.xlsx"
      8 historical = pd.read_csv(historical_path)
----> 9 new_funders = pd.read_csv(new_funders_path)
     11 # Clean column names
     12 historical.columns = historical.columns.str.strip()

File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pandas/io/parsers/readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, dtype, engine, converters, true_values, false_values, skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col, date_parser, date_format, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines, delim_whitespace, low_memory, memory_map, float_precision, storage_options, dtype_backend)
   1013 kwds_defaults = _refine_defaults_read(
   1014     dialect,
   1015     delimiter,
   (...)   1022     dtype_backend=dtype_backend,
   1023 )
   1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)

File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pandas/io/parsers/readers.py:620, in _read(filepath_or_buffer, kwds)
    617 _validate_names(kwds.get("names", None))
    619 # Create the parser.
--> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
    622 if chunksize or iterator:
    623     return parser

File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pandas/io/parsers/readers.py:1620, in TextFileReader.__init__(self, f, engine, **kwds)
   1617     self.options["has_index_names"] = kwds["has_index_names"]
   1619 self.handles: IOHandles | None = None
-> 1620 self._engine = self._make_engine(f, self.engine)

File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pandas/io/parsers/readers.py:1898, in TextFileReader._make_engine(self, f, engine)
   1895     raise ValueError(msg)
   1897 try:
-> 1898     return mapping[engine](f, **self.options)
   1899 except Exception:
   1900     if self.handles is not None:

File /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pandas/io/parsers/c_parser_wrapper.py:93, in CParserWrapper.__init__(self, src, **kwds)
     90 if kwds["dtype_backend"] == "pyarrow":
```

```
     91     # Fail here loudly instead of in cython after reading
     92     import_optional_dependency("pyarrow")
---> 93 self._reader = parsers.TextReader(src, **kwds)
     95 self.unnamed_cols = self._reader.unnamed_cols
     97 # error: Cannot determine type of 'names'

File pandas/_libs/parsers.pyx:574, in pandas._libs.parsers.TextReader.__cinit__()

File pandas/_libs/parsers.pyx:663, in pandas._libs.parsers.TextReader._get_header()

File pandas/_libs/parsers.pyx:874, in pandas._libs.parsers.TextReader._tokenize_rows()

File pandas/_libs/parsers.pyx:891, in pandas._libs.parsers.TextReader._check_tokenize_status()

File pandas/_libs/parsers.pyx:2053, in pandas._libs.parsers.raise_parser_error()

File <frozen codecs>:325, in decode(self, input, final)

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xcc in position 12: invalid continuation byte
```

In [17]:

```python
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

historical_path = "historical_funders_features.csv"
new_funders_path = "new_potential_funders_template_filled.csv"

historical = pd.read_csv(historical_path)
new_funders = pd.read_csv(new_funders_path)

# Clean column names
historical.columns = historical.columns.str.strip()
new_funders.columns = new_funders.columns.str.strip()

# Creating reengage_high if it doesn't already exist
if "reengage_high" not in historical.columns:
    if "reengage" in historical.columns:
        historical["reengage_high"] = (
            historical["reengage"]
            .astype(str)
            .str.lower()
            .str.contains("high")
            .astype(int)
        )
    else:
        historical["reengage_high"] = 0

features = [
    "digital_skills",
    "youth",
```

```
    "women",
    "workforce",
    "entrepreneurship",
    "tajikistan",
    "central_asia",
    "government",
    "foundation",
    "un_agency",
    "development_agency"
]

# Add any missing feature columns as 0
for col in features + ["funded_before", "reengage_high"]:
    if col not in historical.columns:
        historical[col] = 0
    if col in features and col not in new_funders.columns:
        new_funders[col] = 0

strong_historical = historical[
    (historical["funded_before"] == 1) |
    (historical["reengage_high"] == 1)
].copy()

# Backup in case no rows qualify
if strong_historical.empty:
    strong_historical = historical.copy()

historical_matrix = strong_historical[features].fillna(0)
new_matrix = new_funders[features].fillna(0)

similarity_matrix = cosine_similarity(new_matrix, historical_matrix)

new_funders["strategic_fit_score"] = similarity_matrix.mean(axis=1)

closest_match_index = similarity_matrix.argmax(axis=1)

new_funders["closest_historical_match"] = [
    strong_historical.iloc[i]["funder"]
    for i in closest_match_index
]

new_funders["closest_match_score"] = similarity_matrix.max(axis=1)

def recommendation(score):
    if score >= 0.75:
        return "Very strong fit — prioritize soon"
    elif score >= 0.60:
        return "Strong fit — worth follow-up"
    elif score >= 0.45:
        return "Moderate fit — research more"
    else:
        return "Lower fit"

new_funders["recommendation"] = new_funders["strategic_fit_score"].apply(rec

def create_rationale(row):
```

```
    reasons = []

    if row["digital_skills"] == 1:
        reasons.append("digital skills")
    if row["youth"] == 1:
        reasons.append("youth")
    if row["women"] == 1:
        reasons.append("women/gender inclusion")
    if row["workforce"] == 1:
        reasons.append("workforce development")
    if row["entrepreneurship"] == 1:
        reasons.append("entrepreneurship")
    if row["tajikistan"] == 1:
        reasons.append("Tajikistan focus")
    elif row["central_asia"] == 1:
        reasons.append("Central Asia focus")

    if reasons:
        return "Aligned on: " + ", ".join(reasons)
    return "Limited direct alignment based on current features"

new_funders["rationale"] = new_funders.apply(create_rationale, axis=1)

recommendations = new_funders.sort_values("strategic_fit_score", ascending=F

output = recommendations[[
    "funder",
    "strategic_fit_score",
    "recommendation",
    "closest_historical_match",
    "closest_match_score",
    "rationale"
]]

output.to_csv("new_funder_recommendations.csv", index=False)
recommendations.to_csv("new_funder_scoring_full.csv", index=False)

print(output.to_string(index=False))
```

```
                                funder  strategic_fit_score              reco
mmendation                                              closest_historical_mat
ch  closest_match_score                                                 rationale
            Microsoft Philanthropies            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
                    Cisco Foundation            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
         Internet Society Foundation            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
                               KOICA            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
                                JICA            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
           European Union Delegation            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
                                 GIZ            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
                   Aga Khan Foundation            0.638071 Strong fit – worth
follow-up                                                    UNICEF Tajikista
n             1.000000                   Aligned on: digital skills, youth
                           Google.org            0.520983 Moderate fit – res
earch more                                                    UNICEF Tajikist
an             0.816497 Aligned on: digital skills, youth, entrepreneurship
                               UNDP            0.520983 Moderate fit – res
earch more                                                    UNICEF Tajikist
an             0.816497 Aligned on: digital skills, youth, entrepreneurship
Example: Internet Society Foundation            0.451184 Moderate fit – res
earch more Finland MFA – Fund for Local Cooperation (FLC) / Embassy of Finla
nd             1.000000                              Aligned on: digital skills
                                 ILO            0.451184 Moderate fit – res
earch more                   Eurasia Foundation (USAID-funded, SICA programm
e)             1.000000                                   Aligned on: youth
      Example: Mastercard Foundation            0.319036
Lower fit                   Eurasia Foundation (USAID-funded, SICA programm
e)             0.707107                 Aligned on: youth, entrepreneurship
               Mastercard Foundation            0.319036
Lower fit                   Eurasia Foundation (USAID-funded, SICA programm
e)             0.707107                 Aligned on: youth, entrepreneurship
                                 NaN            0.000000
Lower fit Finland MFA – Fund for Local Cooperation (FLC) / Embassy of Finlan
d              0.000000  Limited direct alignment based on current features
```

In [ ]: